From Zero to Fast

Six Problems, Six Data Structures, One Music App

Paulo Sousa

© 2026 Redis Ltd. All rights reserved.

Hi !

Paulo Sousa

Sr. Performance Engineer

Redis © 2026 Redis Ltd. All rights reserved.

THE IDEA

© 2026 Redis Ltd. All rights reserved.

rediscover.io

RedisCover

a music streaming platform

a. **Daily Mixes** personalized playlists for each user
b. **Play Tracking** song play count tracking
c. **Top Songs** most-played songs
d. **Monthly Metrics** distinct listeners per artist
e. **Similar Songs** custom recommendations

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

What are we going to do?

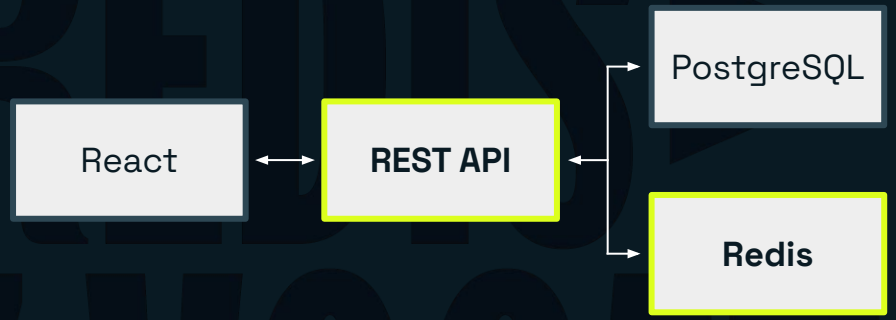
a. Clone the repo and spin up the sample app
b. Identify a problem
c. Measure it
d. Fix it with the right Redis data structure
e. Measure again to confirm the fix
f. Jump to b.

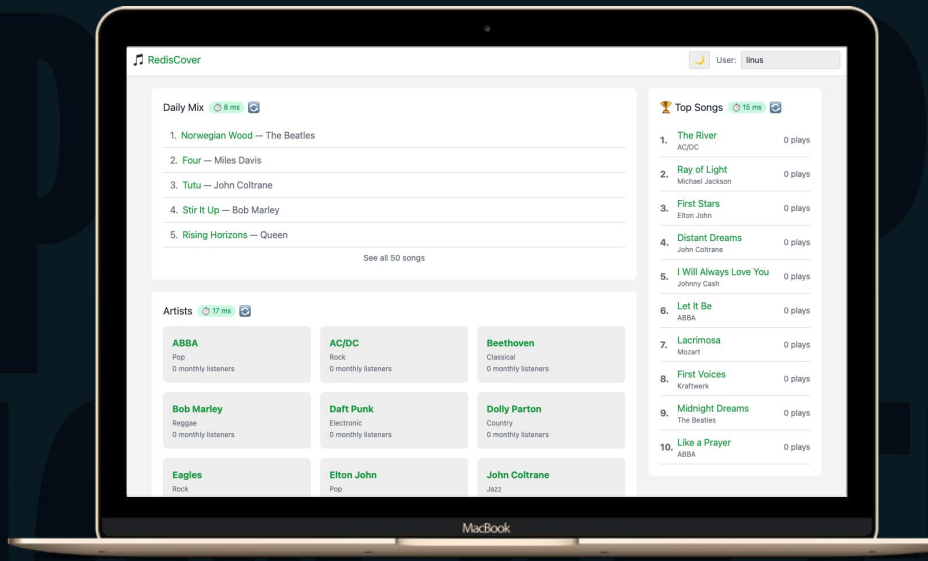
Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Architecture

React + **REST API** + PostgreSQL + **Redis**





© 2026 Redis Ltd. All rights reserved.

`git clone`

https://github.com/paulorsousa/redis-music-workshop

~ `README.md` with all steps ~

Redis © 2026 Redis Ltd. All rights reserved.

1. docker compose up
2. ./workshop health
3. http://localhost:3000
4. Follow the steps on README.md

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Module 1 – Caching

a. Daily mix algorithm is expensive

Check the UI at http://localhost:3000, or via CLI helper:

```
./workshop daily-mix
    (~5 seconds)
```

b. Full algorithm runs every request

c. Compute it once, and cache it on Redis

```
./workshop daily-mix
    (~10 milliseconds)
```

```
git checkout solution/module-1-caching
```

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Module 2 – Atomic counters

a. Concurrent play-count updates via non-atomic SELECT + UPDATE cause lost increments

```
./workshop simulate-plays --song song-1
./workshop simulate-plays --song song-1--concurrent
      Actual:     8 ✗  (92 lost)
```

b. Check the UI: http://localhost:3000

c. Replace SQL calls with Redis INCR

```
./workshop reset
./workshop simulate-plays --song song-1--concurrent
      Actual:     100 ✓
```

❓ Is the “Top Songs” leaderboard working as expected?

```
git checkout solution/module-2-atomic-counters
```

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Module 3 – Sorted Sets

a. We just broke “Top Songs” leaderboard. Check the UI: http://localhost:3000

   Play-count is always zero, even after playing songs. Why?

b. Play-counts are no longer stored in PostgreSQL.

   We need to implement “Top Songs” using Redis.

c. Instead of a simple counter, we need Redis to **keep songs sorted** by play-count

d. Redis Sorted Sets store *elements* ordered by *score*. It also supports atomic score updates :)

e. Follow the steps on README.md

```
git checkout solution/module-3-sorted-sets
```

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Module 4 – Sets

a. Besides every song play, RedisCover also counts each artist's unique monthly listeners

b. Our current solution uses a list for each artist to store their monthly listeners

Problem?
⚠️ Inserting into an unsorted list while enforcing uniqueness is **O(N)** ⚠️

c. Replace the List of unique listeners with a Set (unordered)

d. Follow the steps on README.md

```
git checkout solution/module-4-sets
```

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Module 5 – HyperLogLog

a. Redis Set solved the problem on performance, but…

b. Solution keeps using a lot of memory

   Each artist stores the 36-char ID of each unique listener
   Our Top-5 artists have ~100M monthly listeners
   ⚠️ 36B * 100M * 5 = **18 GB of memory per month** ⚠️

c. HyperLogLog is a probabilistic data structure that **estimates the cardinality of a set**.
   Trades perfect accuracy for efficient memory utilization.
   Redis implementation uses 12 KB of memory (max.) for a standard error rate of 0.81%

d. Follow the steps on README.md

```
git checkout solution/module-5-hll
```

Redis © 2026 Redis Ltd. All rights reserved.

- rediscover.io

Bonus: Module 6 – VectorSets

a. Redis 8.0 introduced a new data type: VectorSets
   Meaning we can now use core Redis as an in-memory **vector database**

b. What makes VectorSets exciting?

⚡ **Memory-native & Fast**	Built on Redis speed and efficiency
🔎 **High-performance search**	Approximate nearest neighbor (ANN) with HNSW (+info)
🏷️ **Rich metadata filtering**	Filter by attributes, tags, or ranges
📈 **Scales with Redis**	Cluster and replication support
🧩 **Easy to use**	Simple commands. No external vector DB needed

c. Great for real-world use cases

🎯 **Recommendations**	Recommended products, content, or users. Example: e-commerce “For you”
🛡️ **Anomaly Detection**	Find unusual patterns in behaviour or transactions. Example: flag suspicious logins
💻 **Code & Dev tools**	Search code by natural language. Example: “Find function to parse JWT”
🤖 **AI APPs & RAG Pipelines**	Store embeddings and run queries against them. Example: chat with your docs

d. Follow the steps on README.md to add “Similar songs” recommendations

Redis © 2026 Redis Ltd. All rights reserved.

QUESTIONS?

sd215ikdlfg05ml8iql0zmdhfrw2mvmlbasj35jkl12pldfu
n457n39dife29f9x5v5g66rga7sew9dcg6yji3lp2a4qwe8r
t9yuip6dfg4hjk4l1mnb2vcx3za4s5d5f68fet4y5h6sd215
ikdl7thankzyou.rw2mvmlbasj35jkl12pldfun457n39dif
e29f9x5v5g66rga7sew9dcg6yji3lp2a4qwe8rt9yuip6dfg
4hjk4l1mnb2vcx3za4s5d5f68fet4y5h6sd215ikdlfg05ml
8iql0zmdhfrw2mvmlbasj35jkl12pldfun457n39dife29f9
x5v5g66rga7sew9dcg6yji3lp2a4qwe8rt9yuip6dfg4hjk4
l1mnb2vcx3za4s5d5f68fet4y5h6sd215ikdlfg05ml8iql0
zmdhfrw2mvmlbasj35jkl12pldfun457n39dife29f9x5v5g
66rga7sew9dcg6yji3lp2a4qwe8rt9yuip6dfg4hjk4l1mnb
2vcx Redis d5f68fet4y5h6sd215ikdlfg05ml8iql0zmdhf
rw2mvmlbasj35jkl12pldfun457n39dife29f9x5v5g66rga
7sew9dcg6yji3lp2a4qwe8rt9yuip6dfg4hjk4l1mnb2vcx3